

Business Rules and Lifecycle

Обновлено 29 мая 2026, 00:41 Андрей Коновалов

Tournament Lifecycle

DRAFT -> ACTIVE -> COMPLETED

|
-> CANCELLED

DRAFT

In this state, the tournament has been created but has not been started.

Allowed:

- join through REST;
- leave for a non-owner participant;
- cancellation by the owner;
- start by the owner when there are at least 4 participants.

ACTIVE

In this state, the live tournament is running.

Allowed:

- submit during the **SUBMISSION** phase;
- vote during the **VOTING** phase;
- receive realtime events;
- get a full snapshot as a participant.

Restrictions:

- a user cannot participate in two active tournaments at the same time;
- leaving through REST is forbidden;

- cancellation is forbidden;
- tournament parameters cannot be edited.

COMPLETED

Final state after the last round.

The backend:

- sets `status = COMPLETED`;
- stores `winnerId`;
- sends `tournament:finished`.

CANCELLED

Final state for a cancelled draft tournament.

The backend:

- sets `status = CANCELLED`;
- deletes participant rows;
- sends `tournament:cancelled`;
- ejects participants from the room.

Round Lifecycle

SUBMISSION

```
-> VOTING / IDLE
-> VOTING / OPEN
-> VOTING / FINALIZING
-> VOTING / OPEN for next submission
-> VOTING / FINISHED
-> completedAt set
```

Main Tournament Rules

Rule	Description
------	-------------

Submission Rules

- Submission is available only to a tournament participant.
- Submission is accepted only during the `SUBMISSION` phase.
- Submission cannot be created or updated after `submissionDeadline`.
- One participant can have only one submission within a round.
- A repeated `POST /rounds/:roundId/submissions` updates the existing submission.
- `content` must be a string with length 1..4000.
- Submission content is not sent in realtime events before voting starts.

The submission phase ends:

- at the deadline;
- or early, if `submittedCount >= totalActiveParticipants` and `totalActiveParticipants > 0`.

Important: `totalActiveParticipants` is calculated from Redis presence, meaning unique online users in the room.

Voting Rules

- Voting starts after the submission phase is closed.
- The backend reveals submissions sequentially, one at a time.
- Votes can be cast only for `currentVotingSubmissionId`.
- The author cannot vote for their own submission.
- A vote can be changed until the current voting window closes.
- A manual vote has `source = MANUAL`.
- Missing votes at the deadline are created as `DISLIKE` with `source = AUTO_TIMEOUT`.
- The submission `roundScore` equals `likeCount`.
- `dislikeCount` is stored and returned in events, but it is not subtracted from the score.

Voting for the current submission ends:

- at `votingDeadline` ;
- or early, when all active eligible voters have voted.

At the deadline, auto-timeout votes are created for all eligible participants, not only online users.

Leaderboard and Winner

After a round is completed, the backend:

1. calculates `likeCount` , `dislikeCount` , and `roundScore` ;
2. stores results in `tournament_round_submissions` ;
3. increments `tournament_participants.cumulativeScore` ;
4. sends `round:completed` ;
5. creates the next round or finishes the tournament.

The leaderboard is sorted by:

1. `cumulativeScore DESC` ;
2. `participant.createdAt ASC` ;
3. `userId ASC` .

The winner after the last round is the first participant from the final leaderboard.